

INTRODUCTION

1. Creational Design Pattern: Singleton

In the development of this advanced Tetris game, certain core management systems, specifically the `GameEngine` and `ThemeManager`, require strict control over their instantiation. The objective is to prevent the creation of multiple game loops or redundant aesthetic configurations running simultaneously, which would lead to critical synchronization issues, resource leaks, and inconsistent game states. To address this, the Singleton creational pattern was selected to guarantee that these management classes have only one globally accessible instance throughout the application's lifecycle.

2. Creational Design Pattern: Prototype

This section analyzes the application of the prototype creational design pattern within the Tetris game implementation. The objective is to demonstrate how the pattern addresses the need for runtime object copying across three subsystems: the ghost piece display, the next piece preview, and the `CustomPieceLoader` distribution mechanism. The pattern is implemented across the `Piece` interface, the `AbstractPiece` base class, all seven standard piece derivatives, the `CustomPiece` class, the `Matrix` class, and the `CustomPieceLoader` module.

In the Tetris game, multiple subsystems require independent copies of an existing piece object at runtime without mutating the original. Direct instantiation using the `new` keyword is insufficient for these use cases, as it always produces a piece in its default state—resetting critical fields such as rotation count and board position. Furthermore, direct instantiation introduces a concrete class dependency: the `GameEngine` would need to know the exact runtime type of the current piece to copy it, violating the Dependency Inversion Principle. The prototype pattern resolves both issues by declaring a `clonePiece()` method in the `Piece` interface, allowing any component to request a stateful copy through the interface alone, with no knowledge of the underlying concrete type.

METHODOLOGY

1. Creational Design Pattern: Singleton

Methodology and Architectural Relationships The Singleton pattern was implemented for the `GameEngine` and `ThemeManager` classes. As explicitly highlighted by the constraint notes in the provided UML diagram (Figure X), this was achieved by declaring `private` constructors for both classes to strictly prevent external instantiation.

The diagram illustrates two distinct approaches to the Singleton implementation tailored to the specific needs of the classes:

1. **GameEngine:** As noted in the UML, it utilizes a `volatile static instance` coupled with a `double-checked locking` mechanism. This ensures absolute thread safety during initialization, preventing race conditions if the engine is called simultaneously by different input listeners or threads managing the game loop.
2. **ThemeManager:** As indicated by its respective note, it employs a straightforward `lazy initialization` approach with a `static instance`, fetching the aesthetic configurations only when first required.

Furthermore, the UML diagram clearly defines how these Singleton instances interact with the rest of the architecture. The `GameEngine` acts as the central controller; it *manages* the `Board` and *uses* both the

GravityAbstraction and ThemeManager to apply environmental effects and visual styles. Crucially, the ThemeManager seamlessly integrates with a Structural pattern (Bridge); it exposes rendering methods (renderEmpty(), renderFilled(), etc.) but strictly delegates the actual execution to the ThemeAbstraction.

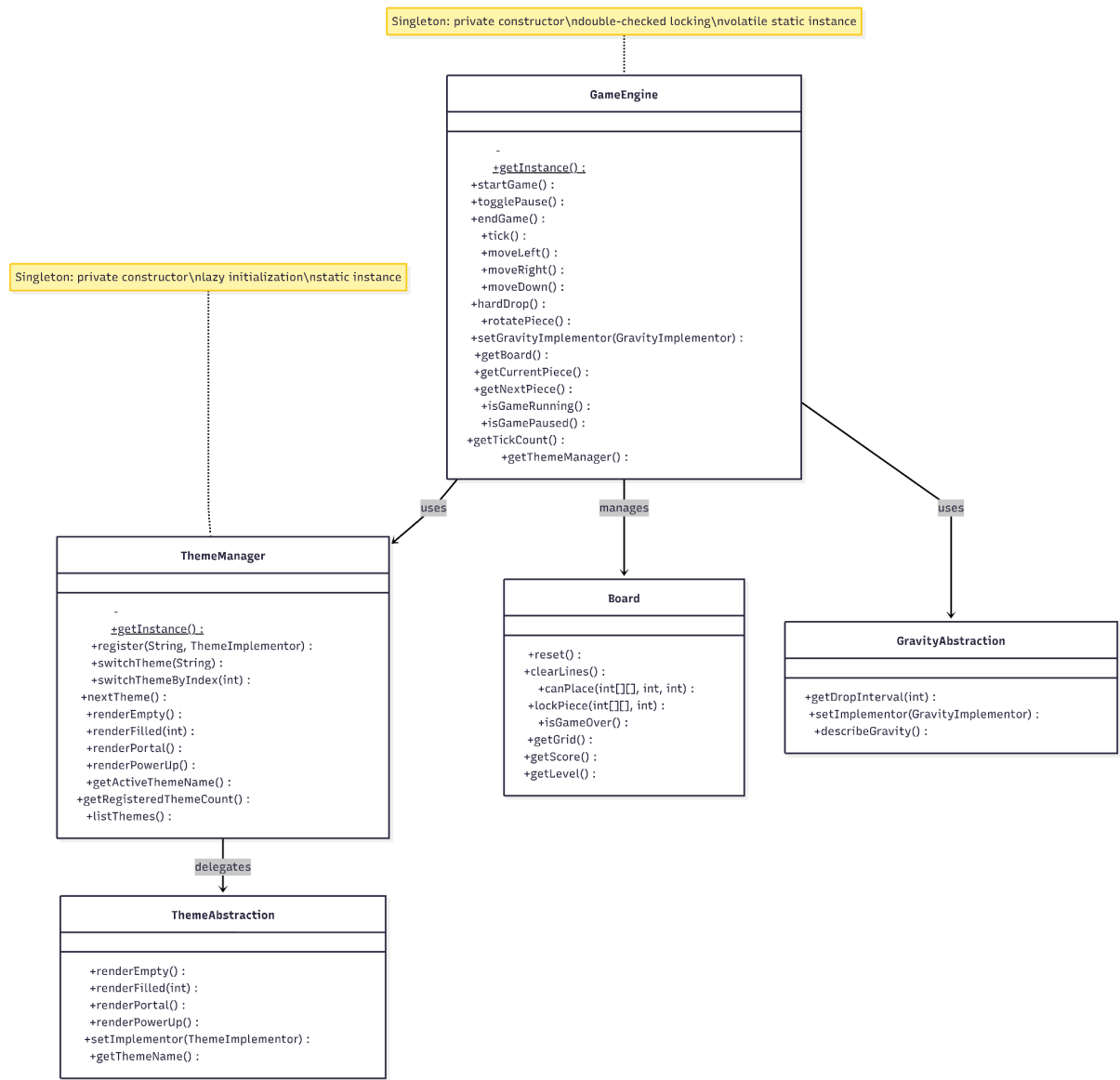


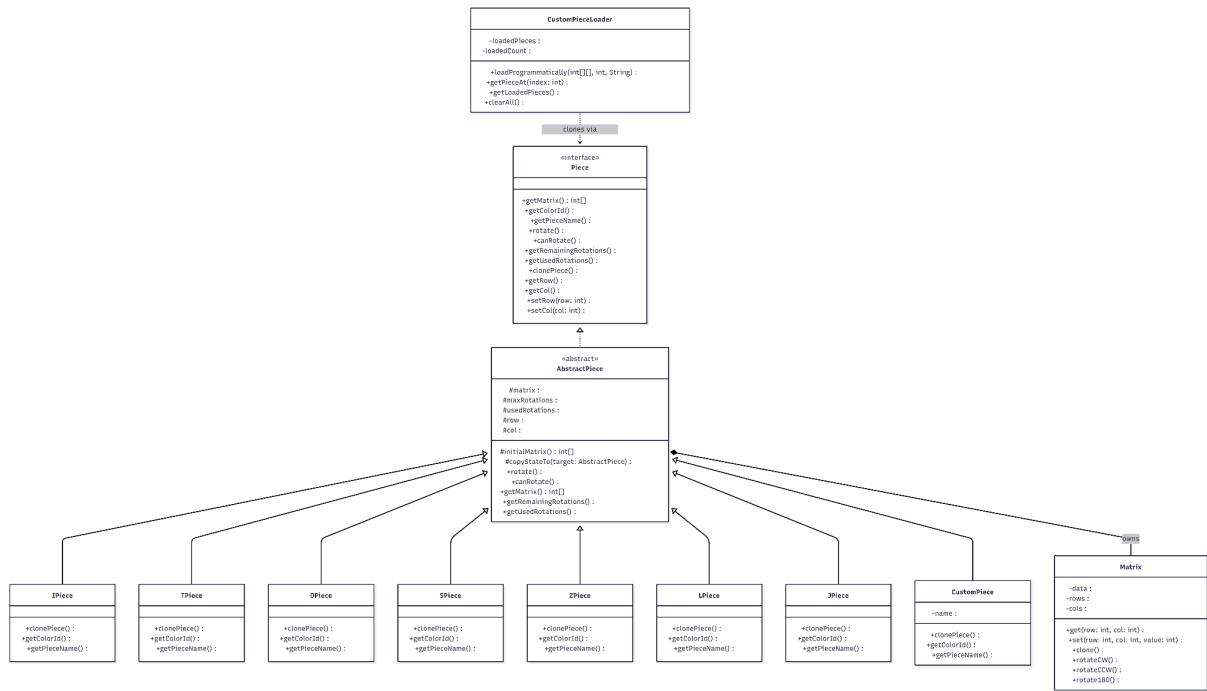
Figure 1: UML Class Diagram detailing the Singleton implementations (GameEngine, ThemeManager) and their structural relationships."

2. Creational Design Pattern: Prototype

The Prototype pattern was selected over alternative creational approaches based on the system's runtime copying requirements. The Factory Method pattern, while present in the codebase through PieceFactory, produces new instances in their initial state and cannot preserve the rotation count or board position of an existing piece. The built-in java.lang.Cloneable interface was deliberately avoided: it does not enforce a typed cloning contract, requires unchecked casting, and does not automatically deep copy composed objects. Since each AbstractPiece owns a Matrix object internally, a shallow copy would cause two piece instances

to share the same data array, meaning a rotation applied to one would corrupt the other. The manual deep copy approach adopted here guarantees full independence between the original and the copy.

The pattern directly supports two SOLID principles. The **Open/Closed Principle** is upheld because adding a new piece type requires only extending `AbstractPiece` and implementing `clonePiece()` — no existing class requires modification. The **Dependency Inversion Principle** is upheld because all client components interact with the `Piece` interface for cloning, remaining agnostic to the concrete type being copied.



IMPLEMENTATION DETAILS

1. Creational Design Pattern: Singleton

Implementation Details (Development Constraints) In strict adherence to the assignment's constraints regarding the prohibition of built-in STL containers (like `ArrayList` or `HashMap`), the Singleton classes were designed using manual data structures.

A notable trade-off is present within the `ThemeManager` class. Normally, mapping a theme's name to its implementation would be handled by a `HashMap`. Since this is forbidden, the `ThemeManager` manually manages theme registration (via its `register()` method shown in the diagram) using fixed-size parallel arrays (`String[]` and `ThemeImplementor[]`). While this manual array-based mapping requires a linear search in the `switchTheme` method, it successfully eliminates the dependency on built-in collections. The Singleton nature of `ThemeManager` guarantees that these custom arrays are initialized only once, providing strictly bounded memory allocation and preventing memory leaks across the game's lifecycle.

2. Creational Design Pattern: Prototype

The implementation separates cloning responsibility into two tiers. At the first tier, the `Piece` interface declares `clonePiece()` as a mandatory contract, ensuring every piece type exposes a uniform cloning entry point. At

the second tier, `AbstractPiece` provides a protected `copyStateTo(AbstractPiece target)` method that centralizes the transfer of all shared state fields — matrix, rotation count, maximum rotations, row, and column — into a single reusable location. Each concrete subclass calls this method within its own `clonePiece()` implementation, avoiding code duplication while retaining the flexibility to override copying behavior independently.

A critical implementation detail is the deep copy performed at the `Matrix` level. Each `AbstractPiece` owns a `Matrix` object that stores piece shape data as a flat array. The `copyStateTo()` method calls `Matrix.clone()`, which allocates a new array and copies every element individually, ensuring the piece copy owns a fully independent matrix. Without this nested clone, two piece instances would remain coupled through shared matrix data, making any in-place mutation on one piece silently corrupt the other.

The `CustomPieceLoader` class manages an internal array of user-defined pieces. Both retrieval methods — `getPieceAt()` and `getLoadedPieces()` — return cloned instances rather than direct references, constituting a defensive copy strategy built on top of the Prototype mechanism. The internal piece pool remains immutable regardless of how the game engine uses or modifies the distributed copies — particularly significant given that custom pieces cannot be reloaded once the input session concludes.

The primary trade-off is the boilerplate introduced by requiring each concrete class to implement `clonePiece()` individually. With eight piece types currently in the system this is manageable, but as the number of variants grows, maintaining consistency requires discipline. A secondary trade-off concerns the `CustomPiece` constructor: because `AbstractPiece`'s constructor calls `initialMatrix()` before the subclass body runs, `CustomPiece` must overwrite the matrix immediately after the `super()` call — a subtle coupling that could introduce bugs if the class is extended without awareness of this constraint.

RESULT AND CONCLUSION

1. Creational Design Pattern: Singleton

Results and Conclusion The implemented Singleton architecture provides a robust, thread-safe, and memory-efficient backbone for the game. The `GameEngine` successfully centralizes the game loop (`tick()`) and movement mechanics, while the `ThemeManager` acts as a unified, globally accessible hub for visual styles. By clearly defining the delegation and usage relationships (as seen in the UML), the system ensures that the core game logic remains entirely decoupled from the visual representation and environmental modifiers, heavily improving overall modularity.

2. Creational Design Pattern: Prototype

The Prototype pattern produces a cloning infrastructure that is consistent and extensible across all subsystems. All components requiring independent piece copies operate through the single `clonePiece()` entry point, with no conditional logic or type inspection at the call site. The two-tier architecture reduces code duplication by consolidating shared state copying in `AbstractPiece.copyStateTo()`. The nested Prototype at the `Matrix` level ensures piece copies are fully self-contained, and the defensive copy strategy in `CustomPieceLoader` successfully isolates the internal piece pool from external modification.

The implementation requires each concrete piece class to explicitly implement `clonePiece()` and call `copyStateTo()`. While the field-transfer logic is centralized, the instantiation step remains distributed across subclasses — a consequence of avoiding reflection to maintain compatibility with course constraints. As the number of piece variants grows, this demands consistent discipline from developers extending the hierarchy.

The Prototype pattern provides a structurally sound solution to the runtime object copying requirements of the Tetris game. By centralizing the cloning contract in the `Piece` interface and consolidating copying logic in `AbstractPiece`, the design adheres to the Open/Closed and Dependency Inversion principles without

sacrificing clarity. The pattern integrates naturally with the Factory Method pattern in `PieceFactory` — the factory handles initial piece creation, while Prototype handles stateful duplication — with each addressing a distinct instantiation scenario. Future extensions such as a Hold piece mechanism or replay system can be built directly on top of the existing infrastructure without modifying any current class.